

Transparent Integration of Non-IP WSN into IP based Networks

Torsten Teubler, Mohamed A. Hail
and Horst Hellbrück

Lübeck University of Applied Sciences
Electrical Engineering and Computer Sciences
Email: {teubler,hail,hellbrueck}@fh-luebeck.de

Abstract—Embedded devices connected to the Internet will start an increasing growth of the Internet in near future. Wireless Sensor Networks (WSN) will play a major role in that growth. In the past several solutions were proposed to make sensor networks IP capable. Today there are IPv6-Stacks available including web servers running on sensor nodes. However, a gateway is always needed to convert the routing protocols and MAC-Layer Protocols including compression of IP packets to run on these devices. The overhead using IPv6 on the nodes is very high in respect of code size and message overhead. Therefore, in our approach we design and implement a system based on simple protocols target for sensor network nodes and a flexible gateway working in a hybrid fashion for our sensor network testbed. We successfully integrated this non-IP WSN in the Internet and our testbed is productive available from any computer connected to the Internet for reference. In this paper we present the architecture of our solution and present the implementation details of a standard WSN application that can be used for evaluation.

Index Terms—Wireless sensor networks; IP networks; Internet

I. INTRODUCTION

Embedded Systems is a continuously growing market for the last decades. With increasing miniaturization and rising performance of integrated circuits and especially microcontrollers more and more products are equipped with intelligence and network interfaces. Wireless interfaces have several advantages for handling of these devices and are an option where space restrictions do not allow to use cables.

This growing market imposes also special technical challenges. One of the challenges is the integration of these smart devices into existing networks or information systems either in the Intranet of companies or in the Internet. In the past several solutions evolved to achieve this integration. We distinguish between solutions based on Application Layer Gateways (ALG) where the sensor network is perceived as an application and approaches that strive for IP connectivity where an IP Stack including Application Layer is needed on the sensor nodes themselves. Both approaches have disadvantages either in the resource limitations of the sensor nodes or in the missing transparency on IP Layer. We will discuss related work in more detail in this paper. Consequently, we suggest a hybrid approach where IP Layer functionality is provided by the gateway and translated to a non-IP protocol in the WSN.

The contributions of this work are the following:

- We present a hybrid approach where our system includes a gateway that acts as an Application Layer Gateway on the one hand and a gateway that provides IP connectivity to non-IP WSNs at the other hand.
- Therefore, we can transparently integrate non-IP WSN to IP networks at all layers including Network Layer.
- We prepared EZgate [1] for productive usage and integrated our WSN protocol AutoCast.
- We present a demonstrator in our testbed to test IP connectivity of our WSN as a reference to users.

The rest of this paper is organized as follows: Section II discusses related work in the field. We introduce our approach in detail in Section III. A first evaluation of our system is presented in Section IV. Section V concludes the paper and presents future work.

II. RELATED WORK

In this section we introduce the two major paradigms for integrating WSNs into IP based networks like the Internet: *Native IP Connectivity* and *Application Level Gateways* (ALG).

A. Native IP Connectivity

The current state-of-the-art approach in connecting WSN natively to the Internet is 6LoWPAN [2] which brings IP to the WSN. With fragmentation and header compression the proposed standard for WSN allows to transmit IPv6 packets via IEEE 802.15.4. 6LoWPAN focuses on IPv6; support for IPv4 is not envisioned.

With IPv6 the MTU (maximum transmission unit) is at least 1280 Byte and in contrast the maximum frame size in IEEE 802.15.4 is 127 Byte. 6LoWPAN is placed between the MAC and the network layer as a protocol translation between the wired Ethernet and IEEE 802.15.4. The translation takes place at a 6LoWPAN gateway.

When using 6LoWPAN a nearly complete IPv6 Stack is needed in the WSN and at least UDP as transport protocol. The advantage is that devices can be addressed directly via IPv6 at the network layer. For a user of a WSN the nodes look like any other networked device as a communication partner.

The point to point communication requires additionally that applications run directly on the WSN devices. Another issue is that WSN nodes have a limited communication range. To form a network and reach all nodes they have to route packets

among them with so called *ad-hoc* routing protocols. On the one hand the 6LoWPAN gateway is a node connected to the Internet or another IP network. On the other hand it is a node as a part in the WSN and therefore it has to be capable of WSNs ad-hoc routing algorithm. Therefore, a native IP connectivity assumes that the gateway is capable of the WSNs ad-hoc routing protocol.

One distinguished implementation is *uIP* from Contiki OS [3]. It covers 6LoWPAN, IPv6, UDP, and TCP for WSN. For evaluation of Contiki's networking stack the *Atmel Raven* evaluation kit is recommended among others. An off-the-shelf PC can serve as 6LoWPAN gateway for Contiki networking stack. The Raven USB stick shipped with the Raven evaluation kit is detected as an Ethernet networking device when using the *Jackdaw* firmware. The most sophisticated application on top of Contiki's networking stack is a web server. With the Raven stick attached to the PC the web pages can be displayed with any common browser on the PC.

Since a standard PC is not capable of an ad-hoc routing protocol only single hop scenario with the above mentioned setup are possible. To the best of our knowledge an implementation of ad-hoc routing on a Raven USB stick is not available yet.

The main disadvantages of this approach are that the WSN nodes have to be capable of running a full IPv6 stack and that IP and the transport protocols create a lot of overhead in the WSN. An IP-Stack is only available for selected types of devices. As the WSN is connected via a gateway converting Ethernet to 6LoWPAN this device has to be capable of a WSN ad-hoc routing protocol additionally. Common approaches using standard PC hardware do not have an appropriate solution to WSN ad-hoc routing.

B. Application Level Gateways ALG

ALGs are a sophisticated approach targeting at application and Web Service integration of WSNs. In ALG approaches the gateway is a web or data base server connected to the Internet and communicates to the WSN via a proprietary protocol. The requests and replies from and to the WSN are fully decoupled from the requests from the Internet. A gateway requests data from the WSN periodically and aggregates or processes data to have current information available to satisfy requests from the Internet. The data presentation of the gateways for the user can be manifold, for example a web page or a Web Service. To the best of our knowledge ALG approaches do not foresee individual communication with the node.

Compared to the native IP connectivity data post processing and preparation is necessary on the gateways as nodes in the WSN are not capable to run neither an IP stack nor applications like web servers. An example of a device type which cannot run complex software due to limited resources are active RFID tags. Attempts to integrate those devices into the Internet are presented in [4] with special ALGs, ALE (Application Level Event Middleware) and EPICS (Electronic Product Code Information System). The ALE is for filtering and processing data from RFID readers and the EPICS provides service logic and data presentation.

WISemid [5] describes a gateway connecting Web Services hosted in the WSN transparently to the Internet. Single nodes in the WSN act as service providers. For a service user service location is transparent and the architecture does not foresee individual addressing of the WSN nodes.

Flexeo [6] proposes a heterogeneous WSN architecture. Constrained devices form a WSN with ZigBee as communication protocol and less constrained devices, so called concentrators acting as gateways connecting the WSN with a web server. A user accesses the web server via HTTP and gets the processed information.

The work in [7] describes a gateway driven approach to connect WSN at service level with the Internet. There are no considerations made how the WSN nodes and the gateway are connected.

Mobile Agents (MA) are an approach to get data in and out a WSN. The MA—a software hosted by the nodes—“jumps” from one node to another while preserving its state and does some work. The communication among the WSN nodes is single hop because the data is collected from the MA. Those approaches are presented in [8] and [9]. An MA also needs a gateway to bring itself or collected data into the Internet.

The authors of [10] state that if the devices have the possibility to run an IP stack it should run on it. Nodes which are not capable of running an IP stack should be attached via gateways. The work distinguishes homogeneous and heterogeneous WSN. In homogeneous networks several gateways should be used to get rid of the single point of failure when having only one gateway. In heterogeneous networks where some nodes are capable of running IP stacks they can act as gateways. To connect the IP addressable nodes in the WSN the authors suggest an *IP overlay network* using the non-IP addressable nodes as relays.

An overlay network is also considered in [11]. This approach is close to our approach. Packets from the Internet to the WSN are encapsulated at the gateway in the proprietary packet format. Unlike our approach every layer has a corresponding layer in the IP/Internet and on the WSN side. It uses virtual nodes which form the gateway logic. How the virtual nodes are able to process the IP packets for the real node in the WSN remains unclear. In contrast our EZgate approach uses *cross layer* techniques to intercept Ethernet frames, IP packets, ICMP messages, UDP packets, or TCP segments and process it and generate messages for the WSN depending on the application.

Our approach is a hybrid solution in the sense that it comprises the two mentioned paradigms IP connectivity and application layer gateway connection. It has the advantage that it can work as an ALG and it can also give a WSN user the experience to work with a natively IP connected WSN. What feature is used belongs to the WSN application. The WSN nodes may provide a web interface for configuring the individual node. To access this web interface a native IP connectivity is desirable. If data has to be aggregated and presented an ALG may be the choice. Our approach also offers advantages regarding *energy consumption* as efficient ad-hoc

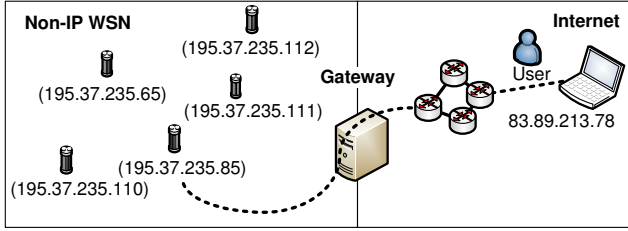


Fig. 1. The Internet integrated (non-IP) WSN from the users perspective. The IP addresses in parentheses are virtual IP addresses managed by the gateway.

routing protocols for the WSN can be chosen arbitrarily. An ALG on top of EZgate has full control on traffic into the WSN and therefore control on WSN energy consumption.

III. CONCEPT

For the integration of non-IP WSN into IP based networks we propose a gateway approach. The goal of our approach is the transparent integration of non-IP WSN in IP networks via a smart gateway. Transparency means that a user of the WSN is unaware of the gateway between the WSN and Internet. Protocols are available transparently for all layers including network layer (e.g. like “ping” nodes via IP). The WSN nodes will be perceived as native IP devices although they are in a non-IP network.

In the case of ALGs the transparency is provided at application level solely. Users may use a web server or a special application to access the WSN through an ALG.

When transparently integrating non-IP WSN IP packets have to be converted into another, maybe proprietary packet format for the ad-hoc routing protocol and vice versa. In the WSN the ad-hoc routing protocol has to provide an addressing scheme for unique identification of WSN nodes (node IDs). The gateway maps IP addresses to node IDs comparable to NAT (Network Address Translation). As the gateway is a part of the WSN it also needs a node ID from the WSN address space.

IP addresses of the WSN nodes are so called *virtual IP* addresses because the WSN nodes are not aware of their IP addresses. Figure 1 shows the WSN from the user perspective: the user can communicate with the WSN nodes like they are natively IP connected. The shown IP addresses are the virtual IP addresses of the nodes the user is aware of. Due to the transparency the user does not know the WSN node IDs used by the ad-hoc routing protocol.

Address resolution with ARP (IPv4) or Neighbor Discovery (IPv6) on the IP side is also provided by the gateway. The gateway serves either as address resolution proxy in the same subnet or as a router in a subnet different from the WSN. Our gateway uses the latter approach.

A. Transparent Gateway

EZgate is an enhanced development of the *user space* TCP/IP protocol stack EZnet [12]. It enables rapid protocol prototyping and fine grained *cross layer* implementations.

EZgate extends EZnet functionality by adding support for multiple—also non-Ethernet—network interfaces as presented in [1]. Thereby we are able to attach EZgate likewise to IP networks and WSN.

EZgate is our proposed approach to connect WSN with IP networks. We do not assume that the WSN has to run some special proprietary protocol. Due to EZgate’s flexibility and rapid development capabilities it is possible to quickly extend EZgate by arbitrary protocols and algorithms.

EZgate provides more than pure protocol conversion by hosting Java applications on top which can access all layers and for IP networks the standard socket API is available. This flexible approach enables rapid development of proxies and ALGs including transparency at the same time when needed.

We believe that forwarding every request from the Internet into the WSN is wasting the limited resources of the WSN nodes. A proxy or ALG with its computational power can process the data from the WSN and answer requests from the Internet faster and more efficient. Energy is considered the most limited resource in a WSN and a proxy also helps to save energy there. Therefore, EZgate is a good option for WSN nodes running an IP stack. Those networks benefit from the potential energy savings as well.

One feature what distinguishes EZgate from other gateways and especially WSN gateway approaches is its *cross layer* functionality. In EZgate intervention at any layer in the stack is feasible. A cross layer application either replaces layers completely or inserts itself underneath or above a layer to process or alter data for that layer. The details for ease of implementation of cross layer applications in EZgate are available in [1].

For the work presented in this paper we implemented a cross layer application which registers itself as a handler for IP packets on EZgate. This application is configured to proxy some selected IP addresses which are mapped to WSN node IDs. The application inspects incoming IP packets. If the packet is addressed to one of the WSN nodes the application forwards an appropriate message for the WSN.

From an IP network or the Internet this looks like WSN nodes *directly attached* to the Internet. With EZgate we can establish transparency at the network layer even with a gateway between the non-IP WSN and the IP network! This gives a user the perception like using Contiki uIP for example: the communication with the nodes can be done with standard software at the PC side e.g. “ping” at the command line. In contrast to the approach of native IP connection with 6LoWPAN we are not restricted to IPv6.

B. WSN Ad-Hoc Routing

In IP networks user data is sent in IP packets. Ad-hoc routing protocols have equivalents to IP packets like messages, frames or data units.

Research in WSN assumes multi hop communication among the WSN nodes. Since our *TriSOS* nodes are capable of running the Wiselib algorithm library [13] we have instant access to many ad-hoc-routing protocols. The WSN gateway

is also a node in the WSN and therefore it has also to be capable of the appropriate ad-hoc routing protocol. Therefore we had to choose a protocol implementation for both the WSN and the gateway.

We already extended Wiselib by the data dissemination protocol *AutoCast* [14] and we have access to an already existing Java implementation of *AutoCast*. The Java implementation of *AutoCast* was designed for use in a simulation environment and it was astonishing to see that this implementation works in a real world deployment with only minor changes. Furthermore *AutoCast* seems to be a good choice because we want to use it in a mobile scenario in the future and *AutoCast* shows promising results in mobile scenarios. By assembling pieces together we quickly obtain a working solution which satisfies future demands.

AutoCast as a *data dissemination* protocol is not a typical ad-hoc routing approach where data is sent from a source to a sink. In *AutoCast* data is sent with a broadcast to all neighbors. One *AutoCast* message can contain multiple application data encapsulated in a so called *DataUnit* (DU). A *DataUnit* is the equivalence of an IP packet in our WSN. An application can send one or more DUs by using the *AutoCast* API method *send()*.

DUs are stored by *AutoCast* and forwarded if needed. To avoid storing DUs infinitely *AutoCast* asks registered applications via the API callback method *getState()* about their state. The state can be *active*, *stale*, *invalid*, or *unknown*. If an application reports an invalid state the DU is deleted. Removal of DUs is important because the maximum number of DUs our WSN nodes can handle is currently 55 due to message size limitations. To achieve expiry of DUs we use time stamps in our DUs together with a time synchronization algorithm described and evaluated in [15] to have the same logical time among the nodes in the WSN. A DU has a hash value which is unique among the DUs currently in the network. The hash values of all stale and active DUs are put in a message. Even when no DUs are to send every node broadcasts a message periodically. This message is called *beacon*. If a node receives a message from a neighbor containing none or a subset of his active DUs it will broadcast a message containing the DUs missing at the neighbor with the intention that the neighbor receives the missing DUs. With beacons the nodes are able to determine their neighborhood. The time when the next beacon is sent is placed in the message to delete neighbors from the neighborhood when the beacon stays away.

A schematic overview depicting the relation between *AutoCast*, API methods, applications, DUs, and message can be found in Figure 2.

This brief overview ignores technical details of *AutoCast*. More information about *AutoCast* can be found in [14] and [15].

As mentioned before the existing *AutoCast* Java implementation was integrated into *EZgate*. Figure 3 gives an overview to the *EZgate* architecture extended by *AutoCast*.

Beginning from the bottom of the figure one can see the network interface devices. On the left side the Ethernet

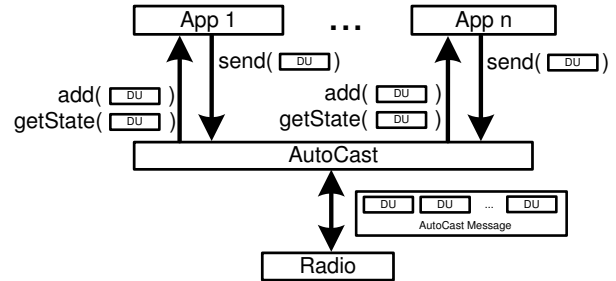


Fig. 2. Schematic Overview of *AutoCast*, API methods, Applications, DUs, Message

network interface of the IP network and on the right side the IEEE 802.15.4 interface connecting the WSN. *EZgate* uses *Libpcap* to fetch Ethernet frames from the Ethernet network interface. To interact with the WSN an IEEE 802.15.4 hardware interface is needed. Here we use the Atmel Raven USB Stick as hardware device which is attached to the PC running *EZgate*. We extended the firmware of the Raven Stick which appears as a serial interface on the PC to send and receive IEEE 802.15.4 frames of maximal size.

Above the Ethernet interface the ARP protocol is depicted. And further there are layers for IPv4, IPv6, ICMP, ICMP6/NDP, TCP, and UDP.

On top of *EZgate* there can be one or more applications. In the figure an example application named *Gateway Application* is depicted. Such an application can use the Java standard Socket API to send and receive data via TCP and UDP. The box *IPv4/IPv6 Packets* with the dashed border on the left side connecting the IP layer with the application depicts that the example application also gets raw IP frames. The *Gateway Application* acts as a gateway because it has access to the IP network as well to the WSN.

On the right side of the *EZgate* stack one can see that the *Gateway Application* is connected to the *AutoCast* layer as well. Here the application can communicate with *AutoCast* via the API proposed in [15]. The *AutoCast Adapter* is used to form IEEE 802.15.4 compliant frames as the internal representation of data in the *AutoCast* implementation differs.

A WSN with three sensor nodes is depicted on the right side of figure Figure 3.

IV. EVALUATION

A. Transparent Gateway Application: IP

A very basic protocol in IP networks is the Internet Protocol itself. IP is the base of node connectivity and the heart of today's Internet. With ICMP we can create IP packets to test IP connectivity. That was the reason to first implement ICMP Echo to proof IP connectivity of our WSN nodes. We will explain the implementation details of ICMP in more detail instead of the core functionality of the IP implementation as we can describe a more detailed scenario.

The ICMP echo request/reply message implementation at the WSN nodes is different from that on *EZgate*. We do

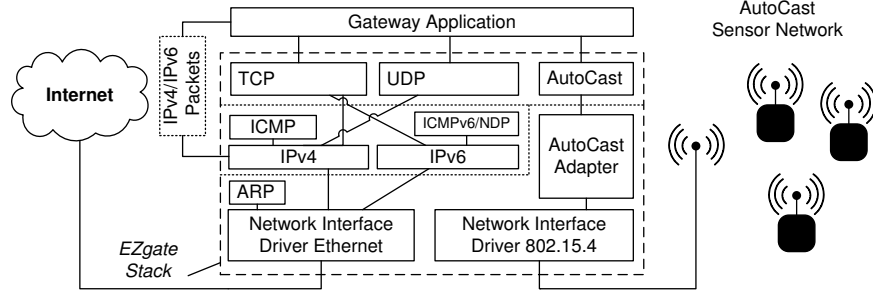


Fig. 3. Gateway architecture

not expect that the nodes issue an ICMP echo request and therefore the implementation for the nodes doesn't cover this feature yet. When an ICMP echo request arrives at EZgate it stores the ICMP echo request in a hash map and generates an *Echo request DU*. The echo request DU contains the type, code, sequence number fields with the ICMP echo packet. In addition the echo DU contains fields for source and destination ID and a hop counter as an equivalent of the time to live (TTL) in IP networks. The echo DUs have an expiration time and therefore they contain fields for storing the time stamp and their maximum life time.

The WSN nodes are not aware of IP addresses. They are addressed by a two byte node ID. On the gateway a mapping from IP addresses to node IDs takes place. In our current implementation the mapping is static by a configuration file. The gateway itself has an IP address for IP and additionally a two byte node ID in the WSN.

If the echo request DU is received by the node whose ID matches the destination ID of the echo request an *echo reply DU* will be created. The destination ID of the echo reply DU is the gateway ID. To count the hops of the echo reply DU in the WSN the field hop count is initialized with the value 128 and is decremented on every intermediate WSN node who receives and forwards the echo reply DU. The behavior is the same as for standard IP packets. If the gateway receives the echo reply DU it generates an ICMP Echo reply with the information from the formerly stored ICMP echo request and sends it back to the issuer of the ICMP echo request. The TTL field in the IP header of the ICMP echo request is set to the value of the corresponding echo reply DU. By that the issuer of an ICMP echo request to a node in the WSN is aware of the hops in the WSN to reach the pinged node.

Transmitting the whole ICMP data to a node in the WSN is questionable due to memory limitations of the nodes. Therefore, we omit this feature for now and the ICMP data is stored at the gateway and resent with the ICMP Echo reply. If transferring huge amount of data within the WSN is needed a potential solution is to spread data among multiple DUs at the gateway.

ICMP echo requests are stored for a certain time interval at the gateway. They are deleted if the echo reply DU arrives from the WSN and if the echo reply DU is not received before

the echo DataUnit expires.

The integration of an AutoCast driven WSN into the Internet demonstrates the flexibility of AutoCast. AutoCast is a content centric data dissemination protocol and IP in contrast is node centric. Our IP implementation can be considered as a node centric overlay on a content centric network.

B. WSN Testbed and Demonstrator

A recent development in evaluation of WSN technology are *testbeds*. Testbeds are a deployment of WSN nodes with an additional management infrastructure independent of the WSN. A testbed infrastructure is based on Ethernet or Wi-Fi. Also common to testbeds is that they use some kind of testbed management software. The most important functionality of that software is reservation of the testbed WSN nodes or parts of it, programming of the testbed WSN nodes and collecting their output for debugging and evaluation.

At Lübeck University of Applied Sciences we deployed a WSN testbed using the WISEBED [16] testbed management software.

Our testbed consists of six stationary infrastructure and seven mobile WSN nodes placed on Roomba robots [17]. Each WSN node is connected to a programmer board and a small-sized PC (FitPC). The FitPCs (see Figure 4) run Windows OS and stationary nodes are attached via Ethernet whereas the mobile nodes use Wi-Fi. A central server is the vital management component of the testbed. The central server and the FitPCs together enable the administration of our testbed.

When the testbed is not in use a *default application* is deployed on the six infrastructure nodes. The default application comprises the demonstrator to test IP connectivity of our WSN, a link quality measurement among the WSN nodes based on the packet loss rate, and a room surveillance with passive infrared sensors. Information provided by the default application in the testbed can be viewed on <http://www.cosa.fh-luebeck.de/en/tools/testbed>. This information is acquired by the testbed management software which reads the debug output of the nodes.

To test the Internet integration of the WSN nodes a user may invoke the command "ping cosa-testbed-fhl1-snx.fh-luebeck.de" where the x can have values from 1 to 6 to select one of the six stationary nodes respectively. The echo requests



Fig. 4. Testbed gateway hardware for EZgate with Atmel Raven Stick in housing attached to front USB port.

of each node from the last 10 minutes can be seen on the testbed website.

As EZgate is a part of the WSN it turns out a good idea to run it on one of the infrastructure FitPCs. The FitPC running EZgate is depicted in Figure 4. At one of the front USB ports the Atmel Raven stick protected in a plastic housing is attached. This gateway FitPC is stationary and has a public Internet address. For administration purposes it is possible to access the FitPC via Wi-Fi.

Nodes have IPv4 addresses exclusively because our university has not switched to IPv6 yet. With minor changes to EZgate's configuration WSN nodes can obtain IPv6 addresses when IPv6 is available at our university.

V. CONCLUSION AND FUTURE WORK

In this work we have presented our proposal for transparent integration of non-IP Wireless Sensor Networks (WSN) in IP networks using the hybrid gateway EZgate. As non-IP WSN protocol we have implemented AutoCast in EZgate for the productive demonstrator. The demonstrator serves as a reference to users to validate IP connectivity of our WSN. With EZgate we were able to quickly implement ad-hoc routing protocols in user space.

We also presented an exemplary gateway application where ICMP runs on top of EZgate using AutoCast as an ad-hoc routing protocol. Herewith, we proved the concept of integrating non-IP WSN in IP networks. The flexibility of EZgate provides operation as an Application Layer Gateway (ALG) and as a transparent gateway for IP connectivity at the same time. A transparent gateway gives a user the perception of a natively IP connected WSN. With our gateway approach WSN can be integrated likewise in IPv4 and IPv6 networks. We believe that this is a clear advantage for the existing heterogeneity in communication networks.

Future extensions include an automatic configuration mode e.g. for mapping of IP addresses to node IDs for WSN nodes. We plan to adapt the auto configuration mechanism in IPv6 to the WSN. Investigations on multiple gateways for one (huge) WSN are of future interest. We avoid single point of failure, achieve load balancing and reduce the number of hops in the WSN.

ACKNOWLEDGMENT

This work was funded by the Federal Ministry of Education & Research of the Federal Republic of

Germany (Förderkennzeichen 01BK0906, GLab) and (Förderkennzeichen 17PNT017, DataCast). The authors alone are responsible for the content of this work.

REFERENCES

- [1] T. Teubler, U. Walther, and H. Hellbrück, "Ezgate - a flexible gateway for the internet of things," in *Proceedings of the Workshops der wissenschaftlichen Konferenz Kommunikation in verteilten Systemen 2011 (WowKiVS 2011)*, T. Margaria, J. Padberg, and G. Taentzer, Eds., vol. 37. Electronic Communications of the EASST, 2011.
- [2] G. Montenegro, N. Kushalnagar, J. Hui, and D. Culler, "Transmission of IPv6 Packets over IEEE 802.15.4 Networks (RFC 4944)," Sep. 2007.
- [3] A. Dunkels, B. Grönvall, and T. Voigt, "Contiki - a lightweight and flexible operating system for tiny networked sensors," in *Proceedings of the First IEEE Workshop on Embedded Networked Sensors (Emnets-I)*, Tampa, Florida, USA, Nov. 2004.
- [4] J. Sung, T. S. Lopez, and D. Kim, "The epc sensor network for rfid and wsn integration infrastructure," *Pervasive Computing and Communications Workshops, IEEE International Conference on*, vol. 0, pp. 618–621, 2007.
- [5] J. Domingues, A. Damaso, and N. Rosa, "Wisemid: Middleware for integrating wireless sensor networks and the internet," in *Distributed Applications and Interoperable Systems*, ser. Lecture Notes in Computer Science, F. Eliassen and R. Kapitza, Eds. Springer Berlin / Heidelberg, 2010, vol. 6115, pp. 70–83.
- [6] J. Vazquez, A. Almeida, I. Doamo, X. Laiseca, and P. Ordua, "Flexeo: An architecture for integrating wireless sensor networks into the internet of things," in *3rd Symposium of Ubiquitous Computing and Ambient Intelligence 2008*, ser. Advances in Soft Computing, J. Corchado, D. Tapia, and J. Bravo, Eds. Springer Berlin / Heidelberg, 2009, vol. 51, pp. 219–228.
- [7] J. Shi and W. Liu, "A service-oriented model for wireless sensor networks with internet," *Computer and Information Technology, International Conference on*, vol. 0, pp. 1045–1049, 2005.
- [8] J. Bai, C. Zang, T. Wang, and H. Yu, "A mobile agents-based real-time mechanism for wireless sensor network access on the internet," *International Conference on Information Acquisition*, vol. 0, pp. 311–315, 2006.
- [9] G. Hackmann, C. Fok, G. Roman, and C. Lu, "Agimone: Middleware support for seamless integration of sensor and ip networks," in *In DCOSS06*, 2006.
- [10] M. Zúñiga Z. and B. Krishnamachari, "Integrating future large-scale wireless sensor networks with the internet," Tech. Rep. CS 03-792, 2003.
- [11] H. Dai and R. Han, "Unifying micro sensor networks with the internet via overlay networking," in *Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks*, ser. LCN '04. Washington, DC, USA: IEEE Computer Society, 2004, pp. 571–572.
- [12] U. Walther and S. Fischer, "Eznet: A framework for rapid protocol prototyping," in *Joint Conference - ICWLHN and ICN 2002 - Networks*. Atlanta, Georgia, USA: World Scientific, 2002, pp. 523–534.
- [13] T. Baumgartner, I. Chatzigiannakis, S. Fekete, C. Koninis, A. Kröller, and A. Pyrgelis, "Wiselib: A generic algorithm library for heterogeneous sensor networks," in *Wireless Sensor Networks*, ser. Lecture Notes in Computer Science, J. Silva, B. Krishnamachari, and F. Boavida, Eds. Springer Berlin / Heidelberg, 2010, vol. 5970, pp. 162–177.
- [14] H. Hellbrück, A. Wegener, and S. Fischer, "Autocast: A general-purpose data dissemination protocol and its application in vehicular networks," *Ad Hoc & Sensor Wireless Networks Journal (AHSWN)*, pp. 91–22, 2008.
- [15] S. Ebers, M. A. Hail, S. Fischer, and H. Hellbrück, "Api for data dissemination protocols - evaluation with autocast," in *The Third World Congress on Nature and Biologically Inspired Computing (NaBIC 2011)*. Salamanca, Spain: IEEE, Oct. 2011.
- [16] H. Hellbrück, M. Pagel, A. Kröller, D. Bimschas, D. Pfisterer, and S. Fischer, "Using and operating wireless sensor network testbeds with wisebed," in *Proceedings of the 10th IEEE IFIP Annual Mediterranean Ad Hoc Networking Workshop*, Favignana Island, Sicily, Italy, Jun. 2011.
- [17] M. A. Hail, J. Pinkowski, T. Teubler, M. Danckwardt, D. Pfisterer, and H. Hellbrück, "Roombanet - testbed for mobile networks," in *Proceedings of the Workshops der wissenschaftlichen Konferenz Kommunikation in verteilten Systemen 2011 (WowKiVS 2011)*, T. Margaria, J. Padberg, and G. Taentzer, Eds., vol. 37. Electronic Communications of the EASST, 2011.